

Intrusion Detection Systems

CPT_S 315: Spring 2026
Cole Wilson and Sean Hodgson**

*(*Equal Contribution)*

Abstract

Network intrusion is a large issue that is on the rise, with ever-increasing stakes and consequences. These intrusions occur when attackers gain access to a network they shouldn't have access to by bypassing security rules. Cybercrime impacts nearly all major industries and careers, making it a very high risk area for companies. In fact, as this report is being written, Instructure (the parent company of the Canvas LMS) is being ransomed by a hacking group called ShinyHunters. This is just one example of how cybersecurity touches our daily lives, whether people are aware of it or not. Cyberattacks are also composed of a constantly evolving threat landscape, forming different signatures and footprints, and often stemming from exploits that are discovered fresh nearly every day. Despite this, nearly all attacks leave behind some form of a trace of intrusion. By training models on behavior of existing and known attacks, we can analyze future network activity and classify them as either standard activity, or potential attacks that warrant further attention.

This project is particularly interesting to the two authors, as we are both interested in cybersecurity as a career. We each have extensive experience working on network security in both real world situations and competition scenarios. Learning to detect network intrusions is directly related to future work we each want to work on, which was the main motivation for choosing this topic. We set out to answer several data mining questions, such as "what types of classifiers best model network intrusion analysis?" and "how accurate is historical network intrusion data in today's modern and evolving world?"

One extremely common dataset that consists of network attacks is the NSL-KDD dataset. NSL-KDD contains updated data from an also well-known benchmark called KDD '99. This work analyzes network activity using k-means clustering, random forest classifiers, and a dynamic threshold-based approach to classification decisions to accurately sort out malicious network activity from the benign. Additionally, we explore XGBoost and other more naive approaches to classification. Finally, we take a look at some of the limitations of our model and potential future limitations and impacts. In the end, our model performs with over 92% accuracy on the NSL-KDD Testing benchmark.

NSL-KDD Dataset

The NSL-KDD dataset is composed entirely of records from the 1999 KDD '99 Dataset. Despite the updated name, the NSL variant does not contain additional data, but instead focuses on reducing duplicate or invalid records. This means that the malware and intrusion data is still somewhat "stuck" in the 1990s, although it still provides a benchmark to test various attack detection methods.

The data is in an ARFF format, and sourced from Kaggle. An interesting thing to note is that there is hardly any information or documentation attached to the dataset, other than an HTML from the original data collectors. This shocking lack of support or documentation also lends credibility to the statement that the dataset is somewhat outdated. Additionally, the distributed ARFF header does not have proper syntax and must be slightly fixed before loading it.

The dataset contains many different features such as root access, error rate, protocol type, service, etc, culminating in the overall classification label of attack/nominal network activity. For those featuresets that are indeed attack activity, an additional classification label is attached that describes the attack method. This method is not used in this research, but provides some interesting debugging information about the root cause of each exploit. The data contains two subsets, the training and testing sets. There are over 125k records for training and more than records 22k focused purely on validation and testing. The focus of the updated NSL dataset was removing redundant records from the previous KDD '99 Dataset, which is the information shown in Figure 0:

	Original Records	Distinct Records	Reduction Rate
Attacks	3,925,650	262,178	93.32%
Normal	972,781	812,814	16.44%
Total	4,898,431	1,074,992	78.05%

Statistics of redundant records in the KDD train set

	Original Records	Distinct Records	Reduction Rate
Attacks	250,436	29,378	88.26%
Normal	60,591	47,911	20.92%
Total	311,027	77,289	75.15%

Statistics of redundant records in the KDD test set

Figure 0: Changes to the KDD Dataset in the NSL-KDD Dataset

Problems and Challenges

Despite the readily available data, there are a couple challenges and potential problems with this project. First of all, network intrusion detection systems must distinguish attacks from normal traffic in real time. This means that whatever model we have must run fast and efficiently on a near-real-time system. Next, detection systems must constantly adapt to new attack types. As mentioned before, new exploits are constantly being developed and released, not to mention the ones no one even knows about yet. Any fully functionally Intrusion Detection System (IDS) must quickly adapt to new, "never before seen" attacks that are completely novel. If the IDS is trained on historical data, it must draw enough conclusions to also correctly extrapolate new systems of exploitation from network data. This means that the (relatively) ancient NSL-KDD dataset, while large and detailed, may not contain enough recent data for more modern attacks.

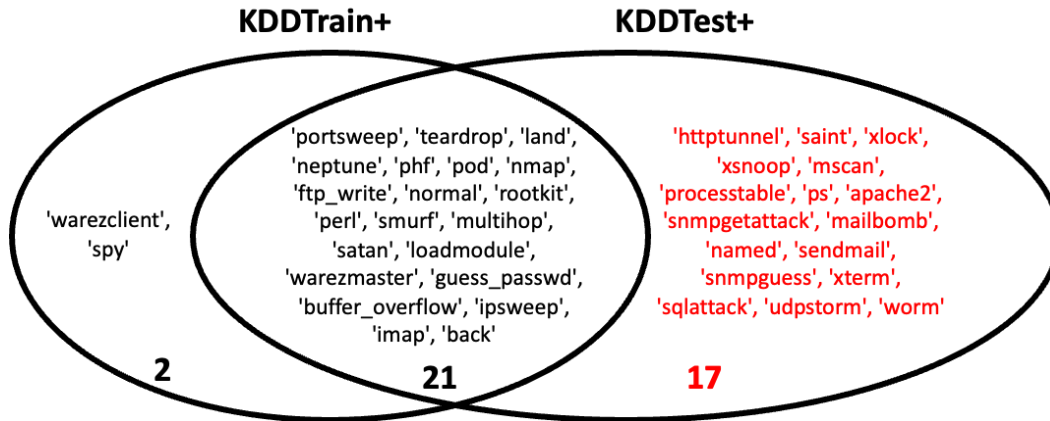


Figure 1: Venn Diagram of Attack Types in Different NSL-KDD Datasets

Another potential issue revolves around how the NSL-KDD sets organize their data. The benchmark contains 17 attack types in the testing/validation set that don't exist in the training data (see Figure 1). While this more accurately represents the way things are in real life, it means that a model's accuracy is typically a lot lower on the testing set than a traditional 80%/20% split on the more homogeneous training data. In fact, most published models with results of over 95% accuracy on NSL-KDD use somewhat

dishonest testing and completely disregard the testing dataset (testing only on a standard 80%/20% split of training data). The KDDTest+ dataset is specifically designed to introduce new attack types that vary from the training data to test a model's accuracy on novel attacks. By disregarding the KDDTest+ set and only training and validating on the KDDTrain+ dataset, you are adding a very important caveat to your results (something these authors hope to avoid).

Initial Approach

First of all, note that all work in this report (regardless of model) was trained on KDDTrain+ and evaluated on KDDTest+ (Rather than an 80/20 on purely KDDTrain+).

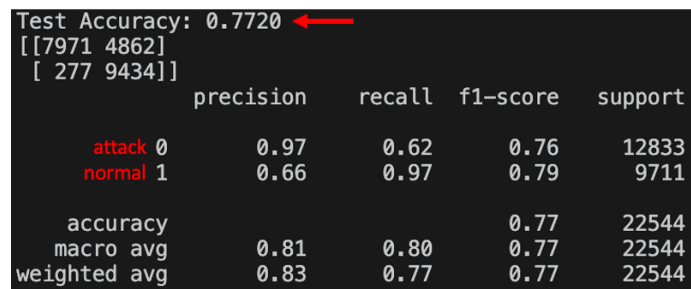
We decided to explore a few different methods of classifiers in order to evaluate what we wanted to continue with. We performed this evaluation without many changes to data scaling or normalization, and instead applied simple label encodings to each categorical field. First, we used a simple Random Forest classification model from PyTorch to simply classify each record as an attack vs normal network usage. Next, we used the XGBoost framework for another classification model. XGBoost performs gradient boosting to combine multiple "weak" branching decision trees into a single coherent model. This is slightly different from a Random Forest model which uses a sparse grouping of distinct trees to achieve the same result.

Using the builtin methods that the Python libraries provided, we used a randomized-search cross validation method for hyperparameter tuning. This is when multiple different combinations of hyperparameters (such as maximum depth, number of estimators, and often much more) are chosen at random and compared against each other to achieve the optimal pairing of parameters.

By using this technique, we constructed an optimal random forest estimator with 413 estimators and a maximum depth of 14 layers, achieving a cross validation score of 0.9985. The XGBoost model was constructed with 249 estimators and a maximum layer depth of 5, gaining a cross validation score of 0.9992.

XGBoost vs Random Forest

We ran the same train/test set on both of the models, with the cross validation hyperparameter tuning randomization code running on each. All in all, they had an identical setup. As seen in the following figures, the test accuracy for the random forest estimator was roughly 7.72%, while the accuracy of the XGBoost trees was roughly 80.4%. Despite the fact that XGBoost had a higher accuracy, both accuracy scores are slightly disappointing (see Figures 2 and 3).

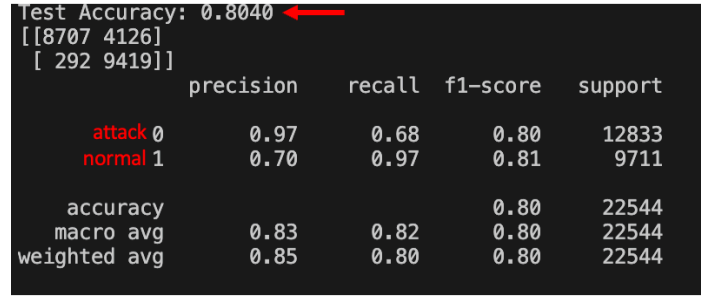


```
Test Accuracy: 0.7720
[[7971 4862]
 [ 277 9434]]
      precision    recall  f1-score   support

   attack 0      0.97      0.62      0.76     12833
   normal 1      0.66      0.97      0.79      9711

   accuracy                0.77     22544
  macro avg      0.81      0.80      0.77     22544
 weighted avg      0.83      0.77      0.77     22544
```

Figure 2: Random Forest Statistics



Test Accuracy: 0.8040				
[[8707 4126]				
[292 9419]]				
	precision	recall	f1-score	support
attack 0	0.97	0.68	0.80	12833
normal 1	0.70	0.97	0.81	9711
accuracy			0.80	22544
macro avg	0.83	0.82	0.80	22544
weighted avg	0.85	0.80	0.80	22544

Figure 3: XGBoost Statistics

For XGBoost, the weighted average of precision was around 85%, while the weighted average of Random Forest was around 83%, which is very similar. We decided that the low accuracy of both showed us that the naïve basic approach of a single decision tree based classifier would not fully adapt and fit to the intricacies of an intrusion detection system. Due to this, we decided that we needed to look into a multi-stage process that involved multiple data mining techniques. Additionally, we would perform better normalization techniques on the data before processing, building a data mining pipeline that would produce better results.

Final Approach

The final approach uses min-max scaling for numerical variables, and one-hot encoding for the categorical values. This enables PyTorch to more precisely represent the variables in a numerical way, and also avoid issues such as when the data is not uniformly distributed and there are strong outliers. This way, an extreme datapoint won't drastically skew the data. This is a new technique that we have not tried to do for the simple random forest classifier or XGBoost model yet, so this should have theoretically boosted our accuracy (and it did).

After passing this preprocessing step, we put the data into a k-means cluster algorithm. This k-means algorithm (at k=9) groups similar data into the same generic cluster. A gross oversimplification of this might be: standard email traffic, online shopping, malicious evil attackers, and watching YouTube. Obviously, the actual higher dimensional algorithm is grouping along more arbitrary connections it makes rather than humanly-understandable groups. However, the overarching concept remains the same: similar activity should appear to be grouped. The clusters are formed with the training data, and then each of the records in the training data is fit into one of the nine clusters. We then devolve into four cases per cluster:

Case 1: If the cluster originally (from the training data before the testing points were introduced) contained less than 1% of "attack" labeled data, the cluster is labeled as nominal. Any additional data that falls into this cluster is also labeled as nominal.

Case 2: Similarly to Case 1, if the data in the cluster is more than 99% attack, the cluster is labeled as an "attack cluster," and once again all future data that falls into that cluster will be instantly labeled as an attack.

Case 3: If the total count of records in the cluster is less than 50 (a very small cluster relatively), it is instantly labeled as an attack. This is based on the assumption that attacks are significantly less prevalent than usual network traffic, and very small groupings of traffic are most likely anomalous and potentially malicious activity.

Case 4: Finally, if an "instant" decision cannot be reached (Cases 1-3), the data goes through two additional steps before final classification: random forest classification and thresholding. This is

the most likely case, as the first three k-means edge cases are primarily to catch clear patterns and very distinct exceptions to the norm.

If Case 4 is reached, then the data is then fed through a random forest classifier with a maximum depth of 40 layers and 600 trees. These models are stored per-cluster, so there are up to 9 individual random forest classifiers within the entire pipeline, although potentially less depending on how many immediate decisions k-means makes. The per-cluster model is trained on the training points within the cluster, and new data points from the testing set will be passed through its predictor in order to generate a confidence of attack.

After a confidence of attack value is predicted, the record is then moved to the thresholding step. What this does, is essentially establish the minimum confidence required to classify a record as an attack. Increasing the threshold value (say, to 0.7 or 0.8), means that there will be many less false alarms, at the expense of potentially missing an actual attack. We will discuss this tradeoff later, in the thresholding analysis section.

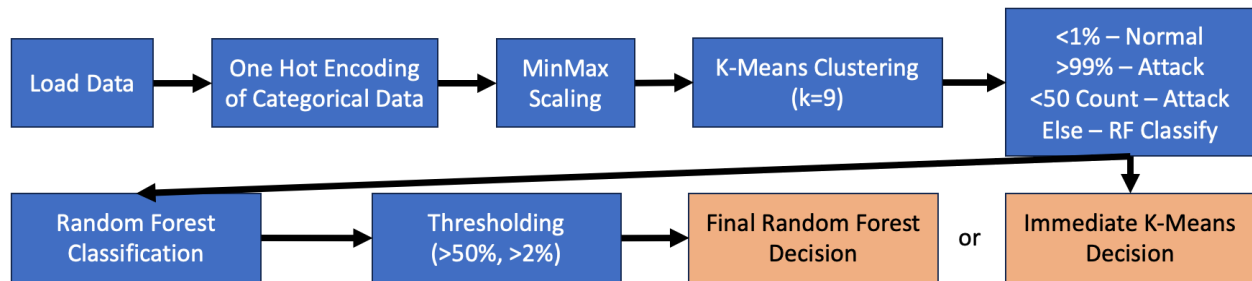


Figure 4: Final Classification Pipeline

For example, let's say that you have a record that is filtered into a group containing fewer than 50 data training datapoints (Case 3). The record is instantly called an attack, and the pipeline execution ends there. The same is true for a point filtered into a cluster containing more than 99% attack data (Case 2), or less than 1% attack data (although it will be labeled as normal: Case 1).

On the other hand, let's say you have a record that doesn't fit any of Cases 1, 2, or 3. In this case, you pass the record through the cluster's random forest classifier and get a prediction of 20% attack. Additionally, let's say your threshold value is set to 50%. Your record is *not* labeled as an attack, because the record is below the threshold of 50%. However, if your threshold was set to 10%, the record would be labeled as an attack as it is greater than 10% confidence.

Thresholding Analysis

Moving on from theoretical analysis, we can now take a look at the true values when run on the KDDTest+ dataset after being trained on the KDDTrain+ dataset. Initially, we tried a threshold of 50%. We saw this as a good starting point, because if the model is more than 50% sure it is an attack, this is a majority (and vice versa). However, we will soon see why we may have been wrong, as we did not yet fully perform a cost/risk analysis of the consequences of a failed intrusion detection. The results for this initial 50% threshold can be seen in the following Figure 5:

```

=====
RESULTS 0.5
Accuracy: 81.12%
F1 Score: 0.8068
Detection Rate: 69.26% (8,888/12,833 caught)
False Alarm Rate: 3.20% (311)

Confusion matrix:
          Predicted Normal  Predicted Attack
Actual Normal      9400         311
Actual Attack     3945         8888

Classification report:
              precision    recall  f1-score   support

   normal       0.70      0.97      0.82     9711
   attack       0.97      0.69      0.81    12833

   accuracy          0.81          0.81    22544
  macro avg          0.84          0.83    22544
 weighted avg          0.85          0.81    22544

per cat recall:
DoS: 0.8538 (6,369/7,460)
Probe: 0.8980 (2,174/2,421)
R2L: 0.1185 (342/2,885)
U2R: 0.0448 (3/67)

```

Figure 5: Results at a 50% Threshold

With a threshold of 50%, we only call a record an attack if we are more than 50% sure of an attack after going through the Random Forest classifier, which can cause significant false negatives. This means that on a uniformly random distribution of attack/no attack classes 50% of them would be reported as attacks. False negatives are bad, as this means an attacker got in and we didn't notice! This is also reflected in the accuracy of around 81%. This accuracy is very similar to the original XGBoost model, so we hope to increase this accuracy significantly rather than keep it at 81%. Additionally, the 50% threshold had a detection rate of only 69.26%, and a false alarm rate of only 3.2%.

While the false alarm rate is quite low, we don't really mind if we have false alarms. While we want to minimize them if possible, it should not be at the expense of other statistics such as accuracy or detection rate. If we were to sell a product that boasted a detection rate of only 69%, it would not get any sales. We hoped to raise that number to over 90%, while maintaining a *relatively* low false alarm rate and a high accuracy as well.

Next, we attempted to try a threshold value of 2%, or 0.02. This is a drastic change, although we want to be very certain that no intrusions occur. In the grand scheme of things, a 2% confidence is still relatively high when you consider it may be a novel attack that has not been seen before, but the model should account for any potential discrepancy. Additionally, the k-means filtering also serves as an extra barrier to preventing accidental misclassifications. In order for records to ever even reach the random forest classifier, they must first be ambiguous enough that k-means was unable to give them a generic instant classification. After performing the same analysis, but with a threshold of only 0.02 (or 2%), we had much much better results!

The results summary for this change is shown below in Figure 6:

```

=====
RESULTS 0.02
Accuracy: 92.49%
F1 Score: 0.9348
Detection Rate: 94.65% (12,147/12,833 caught)
False Alarm Rate: 10.38% (1,008)

Confusion matrix:
          Predicted Normal  Predicted Attack
Actual Normal      8703         1008
Actual Attack      686         12147

Classification report:
              precision    recall  f1-score   support

   normal      0.93      0.90      0.91      9711
   attack      0.92      0.95      0.93     12833

   accuracy          0.92      0.92      0.92     22544
  macro avg          0.93      0.92      0.92     22544
 weighted avg          0.92      0.92      0.92     22544

per cat recall:
DoS: 0.9940 (7,415/7,460)
Probe: 1.0000 (2,421/2,421)
R2L: 0.7795 (2,249/2,885)
U2R: 0.9254 (62/67)

```

Figure 6: Results at a 2% Threshold

Here, we are much more likely to call it an attack, even if we are only 2% or more confident. This gives us many more false positives, but drastically reduces false negatives. Instead of our previous detection rate of only around 69%, we are now seeing detection success as high as 94.6%! Additionally, accuracy has increased by more than 11%, which is significantly better than the previous threshold value of 50%. We would much rather detect false positives and investigate them than miss real incidents.

While the false alarm rate increased to 10.38%, this is likely still very acceptable. Most companies have a team of network security professionals on call at all times, who are more than able to investigate any indecision or false alarms from an automated scanner. It would be much better to investigate a superfluous result than miss a true security incident. If you miss a true security incident, you could potentially be ransomed or lose valuable data, including PII from paying customers. However, if you accidentally get a false positive, the only thing lost is the time spent investigating it.

Confusion Matrices

We can also analyze the confusion matrices for each threshold to visualize the difference slightly better. These display a matrix of predicted vs. actual results in a color-coded matrix. On the horizontal axis is the predicted label (either attack or normal), and on the vertical axis is the ground-truth actual label (also either attack or normal). The color is more saturated and darker blue if the relative frequency of that cell is higher, and more washed out and lighter if it is a lower relative frequency. While this data is also shown in the above figures in text form, seeing it in a visual that also has color is more beneficial to understand the subtle differences between the values.

Below in Figure 7 is the confusion matrix for a threshold of 50%:

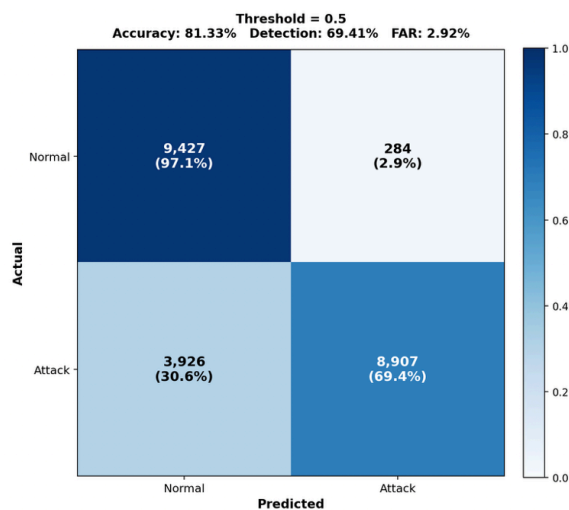


Figure 7: Confusion Matrix for 50% Threshold

You can see that there is a strong navy blue square for Normal/Normal predictions, and a slightly lighter blue square on Attack/Attack predictions. Additionally, the square for false negatives (lower left) is the 3rd darkest, while false positives is practically white. This lines up nearly perfectly¹ with our previous results summary, where false positives were very low, but we had many false negatives. Once again, we would prefer to have false positives over false negatives, because the consequences of false negatives far outweigh those of false positives.

We can also take a look at the confusion matrix for 2% threshold:

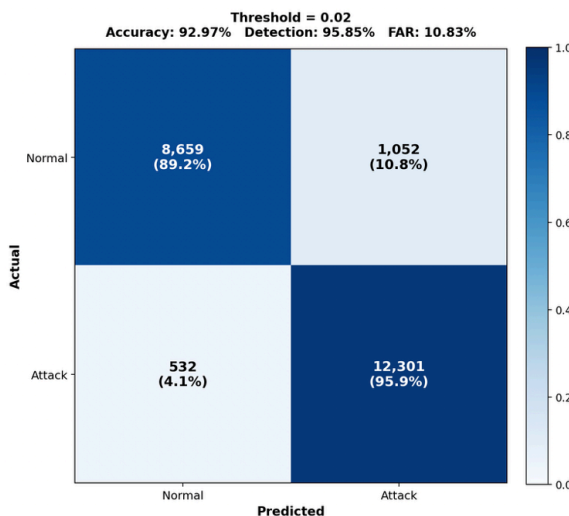


Figure 8: Confusion Matrix for 2% Threshold

Here, we can see a much stronger blue true positive square and much less blue in the false negative square. This is exactly what we want to see, despite the small increase in the upper left false positive

¹ Note that due to non-deterministic algorithms the numbers may slightly vary from earlier summary.

square. Note that each of these matrices is normalized row-wise, which means each row will sum to 100%, and the color indices will sum to 1.0 as well.

Limitations

Despite our success in optimizing the classification pipeline to correctly model network attacks, there are several limitations that the reader should be aware of before actually implementing it in production. As mentioned several times, NSL-KDD is from 2009, and newer attacks have come after that (in fact, nearly every single day after that) which are not represented fully in the data. This means that the model may not accurately defend against a novel attack that it does not have previous information on. Additionally, the k-means $k=9$ number was based on an optimal cluster count for the NSL-KDD data. As new attacks surface, it is very possible that nine clusters will fail to fully categorize enough of the different attacks. This might cause false categorization and drop the accuracy of the model.

Additionally, random forest classifiers may lack the context of regular network activity and business specific patterns. Every company and network is different, with different types of traffic. For example, an IT company might have a lot more "unusual" traffic than, say, a yarn company. Because of this, you still need real humans in the loop, and a generic dataset such as NSL-KDD can't fully fit that.

Also this approach doesn't implement more traditional IP-based bans or virus/malware detection based on content hashes, which is present on many modern computers. Usually, these would stop traffic before it ever gets into the network, which would reduce the load on the model. Currently, this model would be set up and the main entrance to the network, such as in Figure 9. This way, it can analyze all network traffic at once, and serve as a central logging point. However, there are many other ways "into" a network, such as through social engineering or phishing emails. Additionally, employee's phone and personal devices are also an attack vector that needs to be analyzed. The actual data collection can get significantly more difficult on non company-owned and managed devices.

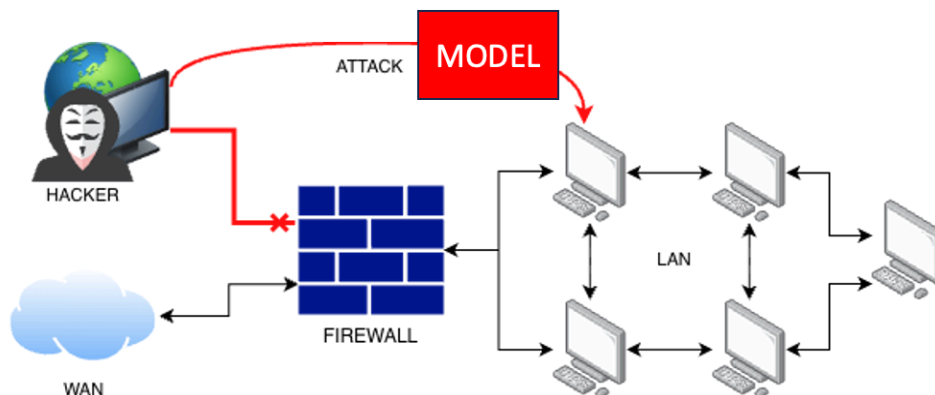


Figure 9: Example Network Diagram

Overall, we've determined that the main limitation with the dataset is that NSL-KDD is extended from a 1999 dataset composed of basic parameters, many of which have changed drastically in today's computing world.

Conclusion

As stated before, we have discovered that false positives are much, much better than false negatives in this scenario. A network security team will always want to investigate potentially anomalous behavior

rather than miss truly malicious behavior. After deciding this, and by reducing our thresholding to around 2%, we were able to drastically boost detection rate with only a small increase in false alarm rate. This is a huge boon to security experts who may use this model, as it allows them to catch so many more incidents than the previous 50% threshold. Despite this boost, we want to reiterate that older datasets such as NSL-KDD are missing much of the required modernization and typical context needed for modern IDS Systems (Intrusion Detection Systems).

We also discovered that more naïve approaches like a pure random forest approach or XGBoost classifier do not fully capture the scenario as well as a thought-out multistage classification pipeline approach. This was made clear by the very low (relatively) accuracy and recall of the two toy test models we created. We also learned that normalizing our data and using effective preprocessing techniques are very necessary for clear and understandable data generation.

We strongly believe that a fully effective intrusion detection system would combine the best of both worlds, such as a fully staffed SOC (Security Operations Center) with staff that could investigate each and every single one of the positive detections, and determine whether or not they are true or false positives. Additionally, this model could be run in conjunction with other security tools such as IP address-based filtering and auditing, along with a zero-trust endpoint security solution for every network endpoint. Finally, the weakest link in any chain is always the human, and this means that phishing prevention and other defenses against social engineering are just as important as more technological approaches like this model based filtering.

Code Reference / Library Credits

There are several files attached to this submission that were used for the generation of figures and training the models and testing them. We used several common Python 3 libraries for data mining and learning such as scipy, pytorch, scikit-learn, numpy, and pandas. Data manipulation was done in numpy and pandas, while pytorch and scikit-learn were used primarily to actually train and test the models.

References

- [1] M. Hassan Zaib, “NSL-KDD Dataset,” www.kaggle.com, Jan. 2019.
<https://www.kaggle.com/datasets/hassan06/nslkdd/data> (accessed Mar. 22, 2026).
- [2] M. Tavallaei, E. Bagheri, W. Lu, and A. Ghorbani, “A Detailed Analysis of the KDD CUP 99 Data Set,” Submitted to Second IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), 2009.
- [3] J. McHugh, “Testing intrusion detection systems: a critique of the 1998 and 1999 darpa intrusion detection system evaluations as performed by Lincoln laboratory,” *ACM Transactions on Information and System Security*, vol. 3, no. 4, pp. 262–294, 2000.